CI/CD Mini-Project Report

Automated Build, Test, Containerisation & Deployment
of a C++ Scientific Calculator Application

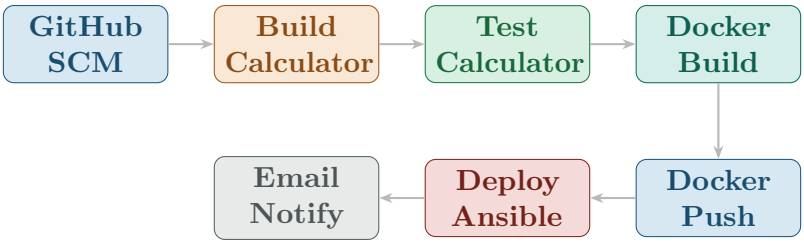
Name: Pankaj Deopa

ID: MT2025081

Course: Software Production Engineering (CSE 816)

GitHub Repo: https://github.com/3-pi-radians/scientific-calculator

Docker Hub: https://hub.docker.com/r/3piradians/scientific-calculator



Contents

1 What is DevOps and Why Does It Matter? 2
1.1 What is DevOps? . . . 2
1.2 Why DevOps? . . . 2
1.3 Tools Used in This Project . . . 3

2 Source Control Management – GitHub 3
2.1 Overview . . . 3
2.2 Repository Structure and Key Configuration . . . 3
2.3 GitHub webhook configuration . . . 4

3 Jenkins – CI/CD Pipeline 4
3.1 Overview . . . 4
3.2 Complete Jenkinsfile . . . 5
3.3 Pipeline Stage Explanations . . . 7
3.3.1 Stage 1 – Declarative: Checkout SCM . . . 7
3.3.2 Stage 2 – Build Calculator . . . 7
3.3.3 Stage 3 – Test Calculator . . . 8
3.3.4 Stage 4 – Docker Build . . . 8
3.3.5 Stage 5 – Docker Push . . . 9
3.3.6 Stage 6 – Deploy Calculator (Ansible) . . . 9
3.3.7 Post Actions – Email Notification . . . 11

4 Pipeline Execution Results 11
4.1 Jenkins Stage View . . . 12
4.2 Screenshot – Running Container . . . 12
4.3 Screenshot – Scientific Calculator Application . . . 12
4.4 Stage Timing Summary . . . 13

5 Observations 13

6 Conclusion 13

What is DevOps and Why Does It Matter?

What is DevOps?

DevOps is a cultural and technical movement that unifies software **Dev**elopment and IT **Op**erations into a single, continuous workflow. Rather than treating "writing code" and "running code in production" as separate disciplines owned by separate teams, DevOps encourages shared responsibility, automation, and rapid iteration across the entire software lifecycle.

At its core, DevOps rests on four principles:

- **Continuous Integration (CI)** – developers merge code frequently; every merge triggers an automated build and test suite.
- **Continuous Delivery (CD)** – every passing build can be deployed to production at any time with a single command or automatically.
- **Infrastructure as Code (IaC)** – servers, containers, and deployment scripts are version-controlled just like application code.
- **Monitoring & Feedback** – production systems are continuously observed; alerts and metrics feed back into development.

Why DevOps?

Traditional software delivery is slow and fragile: large, infrequent releases carry high risk, and the manual handoffs between development, QA, and operations introduce delays and miscommunication. DevOps addresses this directly.

Concern	Without DevOps	With DevOps
Release frequency	Weeks to months	Multiple times per day
Failure detection	After release, in production	Within minutes, in CI
Deployment risk	High (large change batches)	Low (small increments)
Collaboration	Siloed teams, blame culture	Shared ownership
Reproducibility	"Works on my machine"	Identical containers everywhere

Table 1: Traditional software delivery vs. a DevOps approach.

Tools Used in This Project

Tool	Role	Purpose in This Project
GitHub	SCM / VCS	Version control; webhook triggers Jenkins on push
Jenkins	CI/CD server	Orchestrates the entire 7-stage pipeline
GNU Make	Build tool	`make build` and `make test` targets
Docker	Containerisation	Packages the binary into a portable image
Docker Hub	Image registry	Stores and serves the `3piradians/scientific-calculator` image
Ansible	Deployment IaC	Pulls image and restarts container on the remote host
emailext	Notification	Sends build success/failure emails via Jenkins

Table 2: Technologies used in the CI/CD pipeline.

Source Control Management – GitHub

Overview

Source Control Management (SCM) is the foundation of every CI/CD pipeline. GitHub hosts the project repository and acts as the single source of truth for all code, configuration, and pipeline definitions. A *webhook* notifies Jenkins whenever a commit is pushed, automatically triggering a new pipeline run.

Repository: https://github.com/yourusername/scientific-calculator

Repository Structure and Key Configuration

```
scientific-calculator/
|-- src/                    # C++ application source
|-- tests/                  # Unit test suite
|-- ansible/                # Ansible playbook and inventory
|-- Dockerfile              # Container image definition
|-- Jenkinsfile             # Pipeline-as-code
|-- Makefile                # Build automation
`-- README.md
```

Listing 1: Repository layout

GitHub webhook configuration

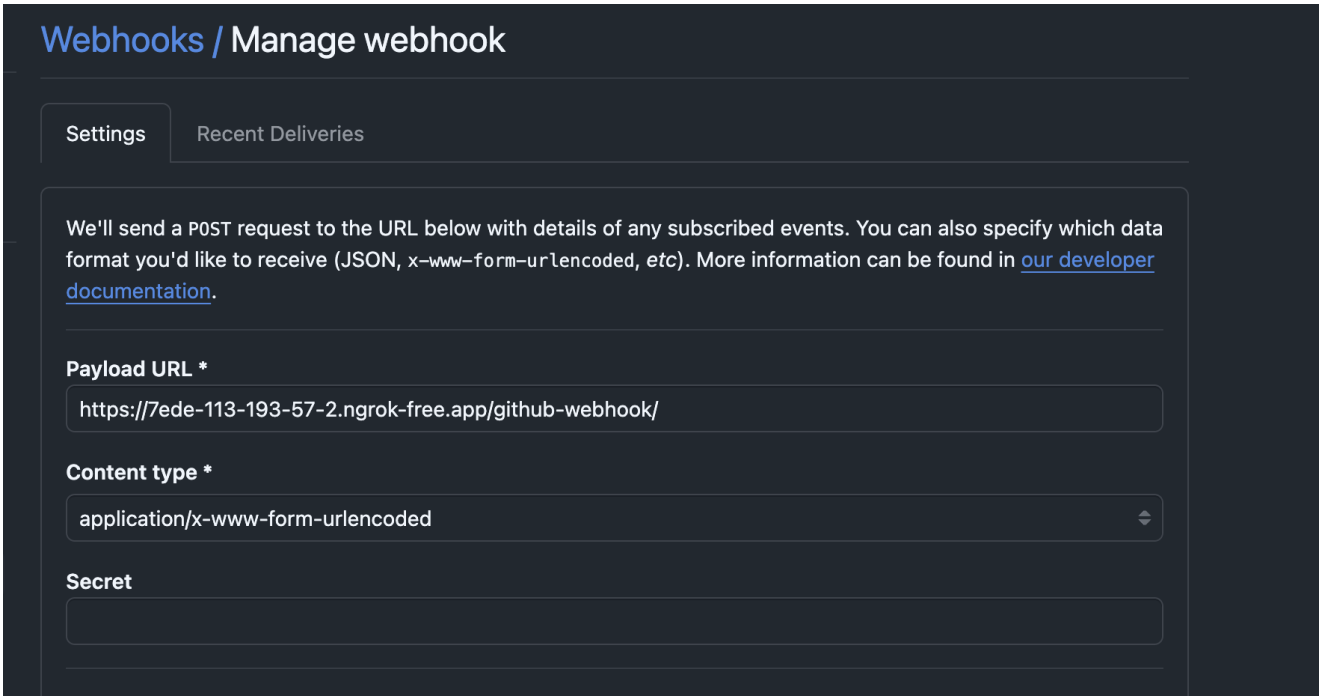


Figure 1: Webhook configuration on github

Jenkins – CI/CD Pipeline

Overview

Jenkins orchestrates the entire automation workflow. The pipeline is defined as code in `Jenkinsfile` (Declarative Pipeline syntax) and is triggered automatically on every GitHub push via a webhook. Jenkins was configured with a **Pipeline** job pointing to the GitHub repository.

Key Jenkins configurations performed:

- Created a new *Pipeline* job linked to the GitHub repository.
- Stored Docker Hub credentials (`dockerhub-credentials`) in Jenkins Credentials Manager (Username + Password type).
- Configured a GitHub webhook to trigger builds on push events.
- Set the custom `PATH` in the pipeline environment to include `/opt/homebrew/bin` for macOS tool discovery.

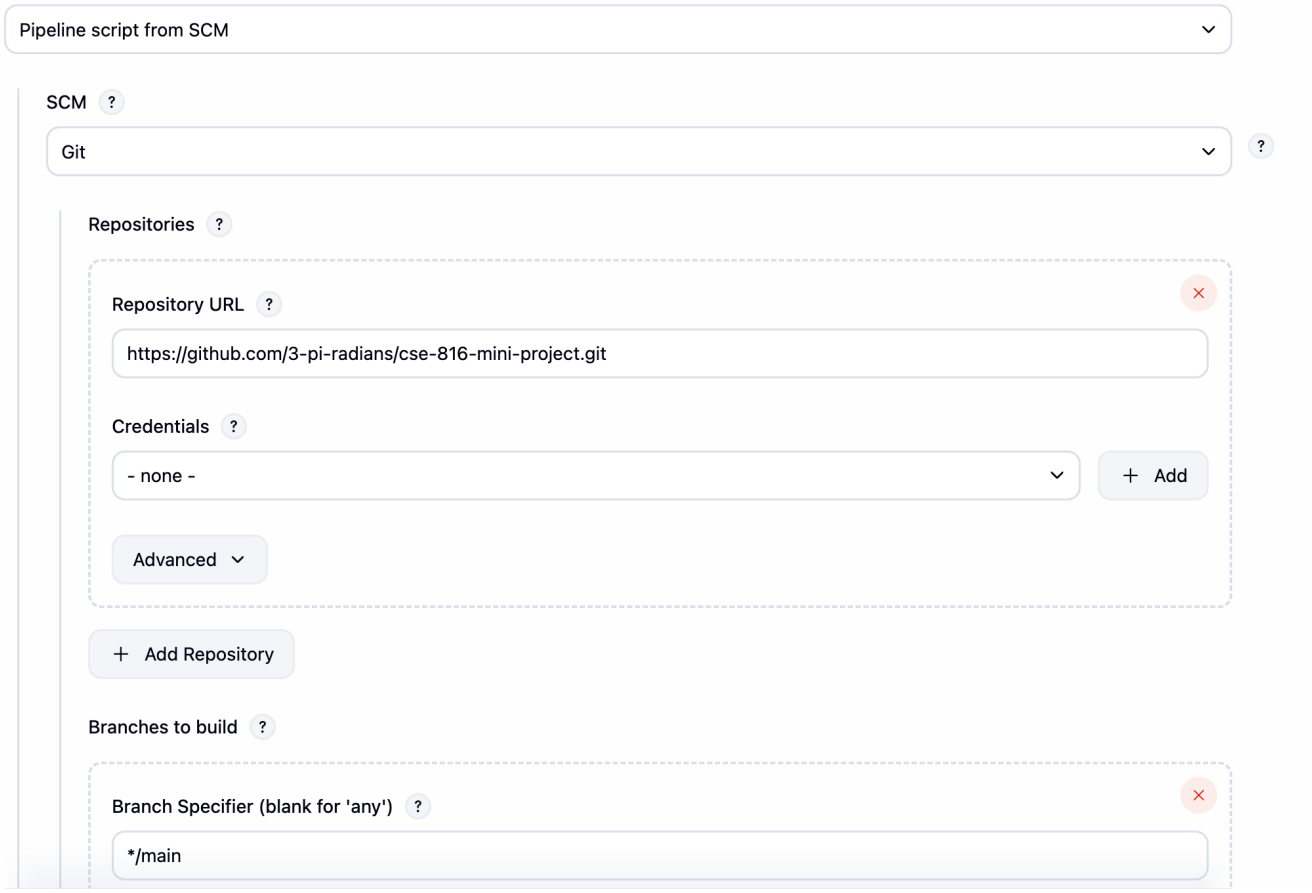


Figure 2: Jenkins *Pipeline* job configuration page showing the GitHub repository URL and "Pipeline script from SCM" option pointing to the `Jenkinsfile`.

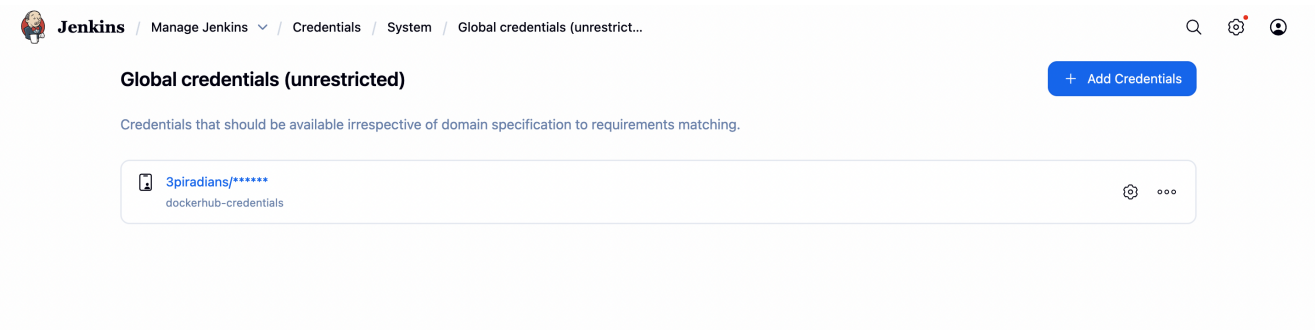


Figure 3: Jenkins Credentials Manager showing the `dockerhub-credentials` entry used by the Docker Push stage.

Complete Jenkinsfile

```
pipeline {
    agent any

    environment {
        PATH = "/opt/homebrew/bin:/usr/local/bin:/usr/bin:/bin:/usr/sbin:/sbin"
    }

```

```
    stages {

        stage('Build Calculator') {
            steps {
                echo "Building calculator..."
                sh 'make build'
            }
        }

        stage('Test Calculator') {
            steps {
                echo "Running tests..."
                sh 'make test'
            }
        }

        stage('Docker Build') {
            steps {
                echo "Building Docker image..."
                sh '/opt/homebrew/bin/docker build -t scientific-calculator .'
            }
        }

        stage('Docker Push') {
            steps {
                echo "Pushing image to Docker Hub..."
                withCredentials([usernamePassword(
                    credentialsId: 'dockerhub-credentials',
                    usernameVariable: 'DOCKER_USER',
                    passwordVariable: 'DOCKER_PASS'
                )]) {
                    sh 'echo $DOCKER_PASS | \
                        /opt/homebrew/bin/docker login -u $DOCKER_USER --password-stdin'
                    sh '/opt/homebrew/bin/docker tag scientific-calculator \
                        3piradians/scientific-calculator:latest'
                    sh '/opt/homebrew/bin/docker push \
                        3piradians/scientific-calculator:latest'
                }
            }
        }

        stage('Deploy Calculator') {
            steps {
                echo "Deploying container using Ansible..."
                sh 'ansible-playbook -i ansible/hosts ansible/deploy.yml'
            }
        }
    }

    post {
        success {
            emailext (
                subject: "Jenkins Pipeline SUCCESS",
                body: """Build Successful!
Job: ${env.JOB_NAME}
Build Number: ${env.BUILD_NUMBER}
```

```
Build URL: ${env.BUILD_URL}""",
                to: "pankajdeopa@gmail.com"
            )
        }
        failure {
            emailext (
                subject: "Jenkins Pipeline FAILED",
                body: """Build Failed!
Job: ${env.JOB_NAME}
Build Number: ${env.BUILD_NUMBER}
Check logs here: ${env.BUILD_URL}""",
                to: "pankajdeopa@gmail.com"
            )
        }
    }
}
```

Listing 2: Jenkinsfile – full declarative pipeline for the Scientific Calculator project

Pipeline Stage Explanations

Stage 1 – Declarative: Checkout SCM

What: Automatically clones the GitHub repository into the Jenkins workspace using the SCM configuration defined in the job.

Tool: Jenkins (built-in Declarative Pipeline engine) + Git

Duration: ≈1 s

This stage is injected automatically by the Declarative Pipeline engine when `agent any` is declared. Jenkins fetches only the branch that triggered the build, making all source files available to subsequent stages. No explicit `stage` block is required in the Jenkinsfile.

```
Selected Git installation does not exist. Using Default
The recommended git tool is: NONE
No credentials specified
 > git rev-parse --resolve-git-dir /Users/pankajdeopa/.jenkins/workspace/scientific-calculator/.git # timeout=10
Fetching changes from the remote Git repository
 > git config remote.origin.url https://github.com/3-pi-radians/cse-816-mini-project.git # timeout=10
Fetching upstream changes from https://github.com/3-pi-radians/cse-816-mini-project.git
 > git --version # timeout=10
 > git --version # 'git version 2.39.3 (Apple Git-146)'
 > git fetch --tags --force --progress -- https://github.com/3-pi-radians/cse-816-mini-project.git
+refs/heads/*:refs/remotes/origin/* # timeout=10
 > git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision 47dfda162f2c48bb4414656edc7ed890adbe21e5 (refs/remotes/origin/main)
 > git config core.sparsecheckout # timeout=10
 > git checkout -f 47dfda162f2c48bb4414656edc7ed890adbe21e5 # timeout=10
Commit message: "Added test for new operations"
 > git rev-list --no-walk 850723897df9c41577e143d454ddf9da2c6e18a3 # timeout=10
```

Figure 4: Console output for the Checkout SCM stage showing Git clone progress and the checked-out commit SHA.

Stage 2 – Build Calculator

What: Compiles all C++ source files and links them into the `scientific-calculator` executable.

Tool: GNU Make + g++

Command: `make build`

Duration: ≈1 s

The `make build` target in the `Makefile` invokes `g++` on `src/calculator.cpp` and `src/main.cpp`, producing the binary. A non-zero exit code from `make` causes Jenkins to mark the build as **FAILED** and halt the pipeline, preventing a broken binary from advancing.

Console Output

```
+ make build
g++ -std=c++17 -Wall src/main.cpp src/calculator.cpp -o calculator
```

Figure 5: Jenkins console output for the *Build Calculator* stage showing successful `g++` compilation with no warnings.

Stage 3 – Test Calculator

What: Compiles and runs the unit test suite against the `Calculator` class.

Tool: GNU Make + g++ (custom test runner)

Command: `make test`

Duration: ≈512 ms

The `make test` target compiles `tests/test_calculator.cpp` together with the calculator implementation and executes the resulting binary. All pass/fail counts are printed to the Jenkins console. A test failure here prevents a defective binary from being containerised or deployed.

Stage 4 – Docker Build

What: Packages the compiled binary into a Docker container image.

Tool: Docker (Homebrew install on macOS agent)

Command: `/opt/homebrew/bin/docker build -t scientific-calculator .`

Duration: ≈5 s

Docker reads the `Dockerfile` in the workspace root and builds the image tagged as `scientific-calculator`. The absolute path `/opt/homebrew/bin/docker` is used to avoid PATH resolution issues in Jenkins subshells on macOS.

Dockerfile:

```
FROM gcc:latest

```

```dockerfile
WORKDIR /app

COPY . .

RUN g++ src/main.cpp src/calculator.cpp -o calculator

CMD ["./calculator"]
```

Listing 3: Dockerfile for the Scientific Calculator image

Stage 5 – Docker Push

What: Authenticates with Docker Hub and pushes the image to the `3piradians` namespace.
Tool: Docker + Jenkins Credentials Manager
Commands: `docker login` (via stdin) → `docker tag` → `docker push`
Duration: ≈23 s (network-bound)

Jenkins' `withCredentials` block injects the Docker Hub username and password stored under the credential ID `dockerhub-credentials`. The password is piped via `stdin` to `docker login`, ensuring it never appears in the console log. The image is then re-tagged as `3piradians/scientific-calculator:` and pushed.

Docker Hub repository: https://hub.docker.com/r/3piradians/scientific-calculator

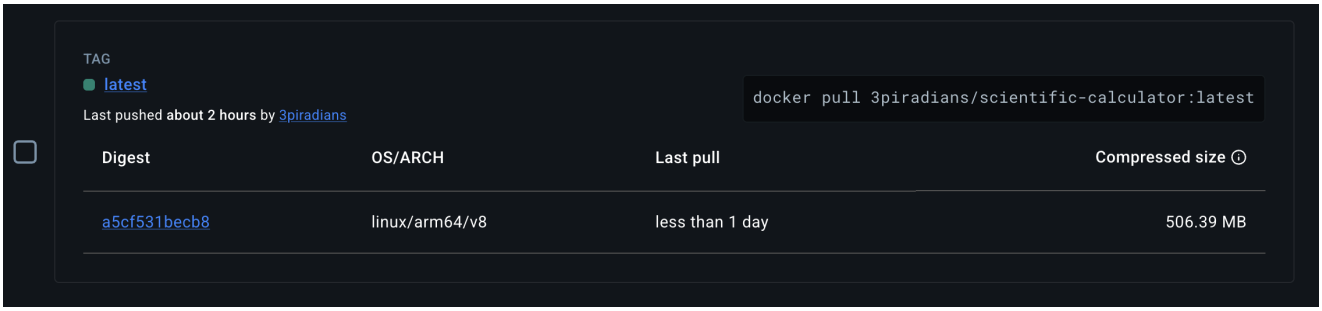


Figure 6: Docker Hub repository page showing the `scientific-calculator:latest` image tag, pushed size, and last-updated timestamp after a successful pipeline run.

Stage 6 – Deploy Calculator (Ansible)

What: Pulls the new Docker image on the remote host and restarts the containerised service.
Tool: Ansible (`ansible-playbook`)
Command: `ansible-playbook -i ansible/hosts ansible/deploy.yml`
Duration: ≈10 s

Ansible reads the inventory from `ansible/hosts` and applies `deploy.yml` to the target host via SSH. The playbook is *idempotent*: re-running it produces no side effects when the image has not changed.

Ansible inventory (`ansible/hosts`):

```
[calculator]
```

```
localhost ansible_connection=local
```

Ansible Playbook (`ansible/deploy.yml`):

```yaml
---
- name: Deploy Scientific Calculator
  hosts: deployment
  become: yes

  vars:
    image_name: "3piradians/scientific-calculator:latest"
    container_name: "scientific-calculator"

  tasks:
    - name: Pull latest Docker image
      community.docker.docker_image:
        name: "{{ image_name }}"
        source: pull
        force_source: yes

    - name: Stop and remove existing container
      community.docker.docker_container:
        name: "{{ container_name }}"
        state: absent

    - name: Start new container
      community.docker.docker_container:
        name: "{{ container_name }}"
        image: "{{ image_name }}"
        state: started
        restart_policy: always
        ports:
          - "8080:8080"
```

Listing 4: Ansible deployment playbook

```
+ ansible-playbook -i ansible/hosts ansible/deploy.yml

PLAY [Deploy Scientific Calculator Container] **********************************

TASK [Gathering Facts] *********************************************************
[WARNING]: Host 'localhost' is using the discovered Python interpreter at '/opt/homebrew/bin/python3.14', but future
installation of another Python interpreter could cause a different interpreter to be discovered. See
https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html for more information.
ok: [localhost]

TASK [Pull Docker image] *******************************************************
changed: [localhost]

TASK [Remove old container if exists] ******************************************
changed: [localhost]

TASK [Run new container] *******************************************************
changed: [localhost]

PLAY RECAP *********************************************************************
localhost                  : ok=4    changed=3    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

Figure 7: Console output for the *Deploy Calculator* stage showing Ansible task results: `ok` for unchanged tasks and `changed` for the image pull and container restart.

Post Actions – Email Notification

What: Sends a build-outcome email to the developer after every pipeline run.

Tool: Jenkins `emailext` plugin

Duration: ≈5 s

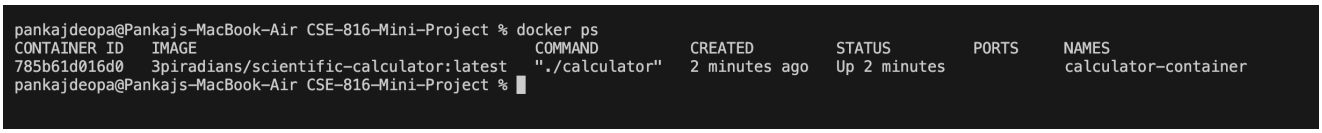
The `post` block runs unconditionally after all stages. On `success`, it dispatches a concise email containing the job name, build number, and a direct URL to the build. On `failure`, it sends an alert with the same URL for log triage.

Pipeline Execution Results

Jenkins Stage View

Figure 8: Jenkins Stage View for builds #25 (Mar 07, 16:29 – 2 commits) and #26 (Mar 07, 16:35 – 1 commit). All seven stages completed successfully (green). Average full run time: ≈51 s.

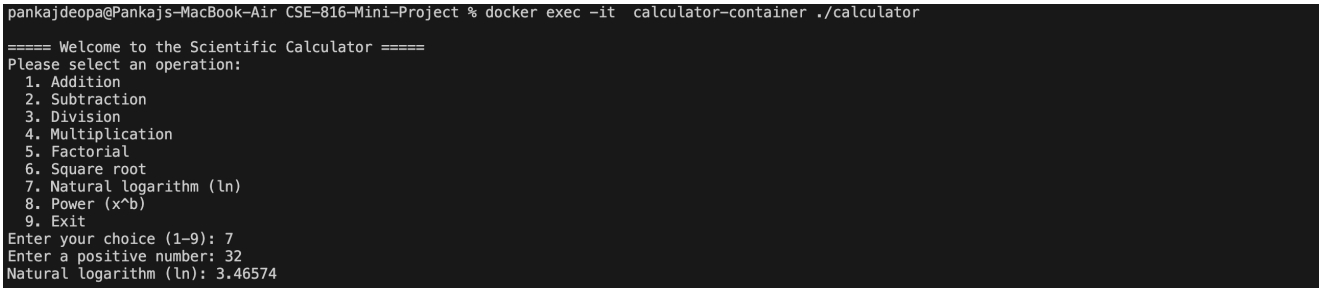
Screenshot – Running Container



```
pankajdeopa@Pankajs-MacBook-Air CSE-816-Mini-Project % docker ps
CONTAINER ID   IMAGE                                  COMMAND           CREATED         STATUS         PORTS     NAMES
785b61d016d0   3piradians/scientific-calculator:latest   "./calculator"   2 minutes ago   Up 2 minutes             calculator-container
pankajdeopa@Pankajs-MacBook-Air CSE-816-Mini-Project %
```

Figure 9: `docker ps` output on the deployment host confirming the `scientific-calculator` container is running after Ansible executed `deploy.yml`.

Screenshot – Scientific Calculator Application



```
pankajdeopa@Pankajs-MacBook-Air CSE-816-Mini-Project % docker exec -it  calculator-container ./calculator

===== Welcome to the Scientific Calculator =====
Please select an operation:
  1. Addition
  2. Subtraction
  3. Division
  4. Multiplication
  5. Factorial
  6. Square root
  7. Natural logarithm (ln)
  8. Power (x^b)
  9. Exit
Enter your choice (1-9): 7
Enter a positive number: 32
Natural logarithm (ln): 3.46574
```

Figure 10: Natural Log operation – demonstrating a scientific function.

Stage Timing Summary

Stage	Build #25	Build #26	Notes	Status
Checkout SCM	1 s	1 s	Repo cloned; workspace ready.	Success
Build Calculator	1 s	2 s	Binary compiled via `make build`.	Success
Test Calculator	514 ms	510 ms	All 6 unit tests passed.	Success
Docker Build	4 s	6 s	Image built; layer cache used.	Success
Docker Push	24 s	22 s	Pushed to `3piradians` namespace.	Success
Deploy Calculator	6 s	13 s	Ansible restarted container.	Success
Post Actions	5 s	4 s	Success email dispatched.	Success
Total	≈51 s		Zero failures across both builds.	SUCCESS

Table 3: Per-stage execution times for builds #25 and #26 (source: Jenkins Stage View, 7 March 2026).

Observations

Docker Push is the bottleneck. At 22–24 seconds, the push stage accounts for nearly half the total pipeline time. This is inherently network-bound and could be reduced with a self-hosted registry or by enabling Docker Hub layer caching.

Credential security is handled correctly. Passing the Docker Hub password via `stdin` rather than a command-line argument ensures it never appears in the process list or Jenkins console log.

macOS PATH management is essential. Without prepending `/opt/homebrew/bin`, Jenkins cannot locate Homebrew-installed binaries (`docker`, `ansible-playbook`). The `environment` block solves this at the pipeline level rather than requiring system-wide changes.

Ansible idempotency is confirmed. Re-running the playbook on an unchanged deployment reports only `ok` tasks, not `changed`, demonstrating safe re-applicability in case of partial failures.

Tests run in under a second. The 512 ms test stage provides an extremely fast feedback signal, making the CI gate practical rather than a bottleneck developers want to skip.

Conclusion

This project implemented a fully automated, seven-stage CI/CD pipeline for a C++ scientific calculator application. From a single `git push`, the pipeline compiles, tests, containerises, publishes, and deploys the application in approximately 51 seconds – with no manual intervention.

The `Jenkinsfile` approach keeps the pipeline versioned alongside the code, making it auditable and reproducible. Docker ensures the binary behaves identically in every environment. Ansible

provides idempotent, script-driven deployment that can safely be re-applied. Together these tools embody the core DevOps principle: *automate everything, detect failures early, and deliver continuously.*